```
{Model} = require './model'

class Post extends Model
  posts = []
  constructor: (@title, @body) ->
    throw 'requires title' unless @title
    super
    @slug = @dirify @title
    posts.push @

  @all: -> posts

  @get: (slug) -> (post for post in posts when post.slug is slug)[0]

  @purge = ->
    posts = []

exports.Post = Post
```

Constructor calls the superconstructor and creates a slug (used for the URL) from the title.

Removes all posts from the internal array.

This Post class assumes post titles are unique. One day this may change, but for now it does the job.

In the following listing you see the base `View` class. This `View` class contains a basic HTML document that can be used to render a response. In a larger program, it's likely that you'd use a template system. In this small example, though, a simple interpolated multiline string does just enough to serve as a template.

Listing 12.8. views/view.coffee

```
class View
  render: ->
    'Lost?'

  wrap: (content) ->
    """
    <!DOCTYPE html>
    <html dir='ltr' lang='en-US'>
    <head>
    <meta http-equiv='Content-Type' content='text/html; charset=utf-8'>
    <title>Agtron's blog</title>
    #{content}
    """

exports.View = View
```

Only a single (very simple) page template is required so far. Keeping it here is sufficient. If the application grows, you'll need to look for another solution.

The next listing shows the `List` class that inherits from `View`. This `List` class is used to render a list of posts.

Listing 12.9. views/list.coffee